Graph-Aware Stance Detection for Syncialo

A Research-Oriented Technical Report on Architecture, Features, Training, and Evaluation

Technical research report

April 10, 2026

Abstract

This document presents a research-style analysis of a graph-aware stance detection model for debate-graph relation classification. The model addresses three-way directed relation prediction between claims, where each candidate edge must be classified as `support`, `attack`, or `neutral`. The approach combines two pretrained transformer-based subsystems, namely a natural language inference model and a target-dependent stance detector, with a debate-local graph feature extractor built from TF–IDF similarity structure. These heterogeneous signals are fused into a thirteen-dimensional representation and classified with a Random Forest. This report details the mathematical structure of the features, the debate-local graph construction process, the training protocol, the benchmark methodology, and the empirical comparison against older baselines and the earlier DeBERTa + stance logistic regression-regression model. It also includes architecture figures, metric bar charts, and confusion matrix visualizations intended to make the model suitable for exposition in a research-paper context.

1 Introduction

The project already contained two important ingredients:

1. strong text-pair base models for natural language inference and target-dependent stance detection, and
2. graph-centric downstream tooling for exporting debate graphs, visualizing them, and running discourse-oriented graph algorithms.

The graph-aware model was introduced to bridge these two parts. Instead of treating edge classification as a purely local sentence-pair problem, the model augments pairwise transformer features with per-debate structural and similarity-based graph features. In effect, the model asks not only "Does claim u support or attack claim v?" but also "How are u and v positioned inside the semantic neighborhood of the debate graph?"

2 Problem Formulation

Each example corresponds to a directed candidate relation between two claims from the same debate:

$$e = (u, v, y) \tag{1}$$

where:

- u is the source claim,
- v is the target claim,

- $y \in \{\texttt{support}, \texttt{attack}, \texttt{neutral}\}$.

The code uses the following text-pair convention:

- **premise** = target claim text,
- **hypothesis** = source claim text.

This means the model predicts the relation from the current source claim to the target claim while using the target as the reference statement in both the NLI and stance components.

3 Method

3.1 Notation

Symbol	Meaning
u	source claim in a directed candidate relation
v	target claim in a directed candidate relation
$e = (u, v, y)$	labeled candidate edge
y	gold label in $\{\texttt{support}, \texttt{attack}, \texttt{neutral}\}$
$\mathcal{C} = \{c_1, \ldots, c_n\}$	set of all claims in one debate
$\mathbf{t}_i$	TF–IDF vector of claim c_i
$N_k(c_i)$	retained top-k similarity-neighbor set of claim c_i
k	neighborhood size limit in the similarity graph
τ	similarity threshold for retaining neighbors
$\mathbf{x}_{\text{NLI}}$	3-dimensional NLI logit vector
$\mathbf{x}_{\text{stance}}$	3-dimensional stance logit vector
$\mathbf{x}_{\text{graph}}$	7-dimensional graph feature vector
$\mathbf{x}_{\text{full}}$	fused 13-dimensional feature vector
$\hat{\mathbf{p}}(\mathbf{x})$	predicted class probability vector
$\hat{y}$	predicted class label

3.2 Dataset and Label Construction

3.3 Dataset Source

The current graph-aware benchmark uses the Syncialo raw dataset:

- repository id: `DebateLabKIT/syncialo-raw`
- corpus: `synthetic_corpus-001`
- split: `eval`

Each debate file is a node-link JSON graph. The project parses:

- `nodes` to obtain claim texts,
- `links` to obtain gold `PRO`/`CON` relations.

3.4 Gold Labels

Gold labels are mapped as follows:

$$\texttt{PRO} \mapsto \texttt{support}$$
$$\texttt{CON} \mapsto \texttt{attack}$$

To train a three-way classifier, the benchmark additionally samples non-edges from the same debate graph and labels them as `neutral`. The current configuration uses:

$$\texttt{neutrals_per_pos} = 1.0$$

so the intended class mixture is approximately one neutral example for every positive edge, subject to sampling availability.

3.5 Per-Debate Processing

All graph-aware feature construction is local to an individual debate. This is a crucial design decision:

- candidate source–target pairs are formed within the same debate only,
- semantic neighborhoods are built only from claims within the same debate,
- graph features are not allowed to connect claims across debates.

However, the final classifier is trained across multiple debates to improve generalization.

3.6 Base Transformer Components

3.6.1 Base NLI Model

The NLI feature block is produced by a Hugging Face sequence classifier loaded from:

`MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli`

Its output logits are canonicalized to the three-dimensional vector:

$$\mathbf{x}_{\text{NLI}} = [\ell_{\text{contradiction}}, \ell_{\text{neutral}}, \ell_{\text{entailment}}] \tag{2}$$

3.6.2 Base Stance Model

The stance feature block is produced by a target-dependent stance classifier loaded from:

`krishnagarg09/stance-detection-semeval2016`

It returns logits in the canonical order:

$$\mathbf{x}_{\text{stance}} = [\ell_{\text{against}}, \ell_{\text{none}}, \ell_{\text{favor}}] \tag{3}$$

In the implementation, the tokenizer input order is:

$$(\text{claim}, \text{target})$$

which, under the project convention, becomes:

$$(\text{source claim}, \text{target claim})$$

3.6.3 Meta Baseline

Before the graph-aware model was added, the main learned combination layer was a multinomial logistic regression classifier over the concatenated six-dimensional transformer feature vector:

$$\mathbf{x}_{\text{base}} = [\mathbf{x}_{\text{NLI}}\ ;\ \mathbf{x}_{\text{stance}}] \in \mathbb{R}^6 \tag{4}$$

This remains the main non-graph baseline for comparison.

3.7 Graph-Aware Feature Design

3.7.1 Motivation

The six transformer logits are informative but fundamentally local: they encode how the source and target claims interact as a pair, but they do not encode how those claims are situated inside the rest of the debate. In structured discourse, this omission matters. Claims that are semantically central, claims that share neighborhoods, and claims that occur inside overlapping topical regions may exhibit relation patterns that are not recoverable from pairwise text alone. The graph-aware model therefore augments the local transformer evidence with seven debate-local graph features derived from a similarity graph constructed over all claims in the same debate.

3.7.2 TF–IDF Construction

The debate-local graph begins with a sparse lexical-semantic representation of every claim in one debate. Let a debate contain claims $c_1, \ldots, c_n$. For a vocabulary term t and document claim c_i, the term frequency component is:

$$\mathrm{tf}(t, c_i) = \text{number of occurrences of term } t \text{ in } c_i \tag{5}$$

In standard vectorized form, one often uses a normalized or sublinear version of this count, but the practical effect is the same: more salient terms in a claim receive larger local weights.

The inverse document frequency component is:

$$\mathrm{idf}(t, \mathcal{C}) = \log\left(\frac{1 + |\mathcal{C}|}{1 + |\{c \in \mathcal{C} : t \in c\}|}\right) + 1 \tag{6}$$

where $\mathcal{C}$ is the set of claims in the current debate. The debate-local TF–IDF weight is then:

$$\mathrm{tfidf}(t, c_i, \mathcal{C}) = \mathrm{tf}(t, c_i) \cdot \mathrm{idf}(t, \mathcal{C}) \tag{7}$$

In the implementation, TF–IDF is computed *per debate*, not across the entire dataset. This design choice is important. The vocabulary statistics are therefore conditioned on the current debate only:

- if a term is rare within one debate, it becomes salient in that debate,
- if a term is common across many claims in the same debate, its discriminative value decreases.

This local fitting behavior is appropriate for graph-aware relation modeling because the objective is to recover neighborhood structure inside each debate rather than to build a global corpus-wide retrieval system.

The implementation uses:

- English stop-word removal,
- `max_features = 4000`,
- a separate TF–IDF fit for every debate.

3.7.3 Similarity Graph Construction

Let $\mathbf{t}_i$ denote the TF–IDF vector for claim c_i. Pairwise similarity is computed via cosine similarity:

$$\mathrm{sim}(c_i, c_j) = \cos(\mathbf{t}_i, \mathbf{t}_j) = \frac{\mathbf{t}_i^\top \mathbf{t}_j}{\|\mathbf{t}_i\|_2 \|\mathbf{t}_j\|_2} \tag{8}$$

For each source claim c_i, the system ranks all other claims by decreasing cosine similarity and retains only neighbors satisfying both conditions:

1. $\text{sim}(c_i, c_j) \geq \tau$
2. the neighbor is among the top-k most similar claims

For the saved benchmark:

$$k = 8, \qquad \tau = 0.15$$

This produces a directed local neighborhood set:

$$N_k(c_i) = \{c_j : c_j \text{ retained as one of the top-}k \text{ neighbors of } c_i\} \tag{9}$$

3.7.4 Formal Feature Vector

For a directed candidate edge (u, v), the graph-aware feature extractor returns:

$$\mathbf{x}_{\text{graph}}(u, v) = \Big[f_1(u,v) \quad f_2(u,v) \quad f_3(u,v) \quad f_4(u,v) \quad f_5(u,v) \quad f_6(u,v) \quad f_7(u,v) \Big] \in \mathbb{R}^7 \tag{10}$$

3.7.5 Feature 1: Cosine Similarity

The first feature is the TF–IDF cosine similarity between the source and target claims:

$$f_1(u, v) = \text{sim}(u, v) \tag{11}$$

This feature measures lexical-semantic proximity inside the current debate. It is useful because relations such as support frequently arise between semantically close claims or paraphrastic elaborations, while very weak similarity can be a useful cue for neutrality. At the same time, attack edges can also retain moderate similarity because rebuttals often reuse the same topic words. Thus this feature is informative but not sufficient on its own, making it a good interaction feature for the downstream classifier.

3.7.6 Feature 2: Normalized Source Degree

Let $N_k(u)$ be the retained similarity-neighbor set of source claim u. The normalized source degree is:

$$f_2(u, v) = \frac{|N_k(u)|}{\max(1, k)} \tag{12}$$

This quantity measures how broadly connected the source claim is in the local semantic graph. A source claim with many similarity neighbors is often topically central or semantically generic. Such claims may behave differently from highly specific claims that attach to only a few neighbors. The feature helps the model determine whether the source is structurally central, potentially affecting the prior likelihood of support, attack, or neutral relations.

3.7.7 Feature 3: Normalized Target Degree

Similarly, the normalized target degree is:

$$f_3(u, v) = \frac{|N_k(v)|}{\max(1, k)} \tag{13}$$

This feature measures how broadly connected the target claim is inside the debate-local similarity graph. A highly connected target may represent a central motion, a common theme, or a semantically broad claim that attracts many related statements. Such central targets often participate in relation patterns that differ from those of isolated targets. Including both source and target degree terms allows the model to distinguish local relation types conditioned on the structural status of both endpoints.

3.7.8 Feature 4: Target-in-Source Neighbors

This binary indicator tests whether the target appears in the source neighborhood:

$$f_4(u, v) = \mathbf{1}[v \in N_k(u)] \tag{14}$$

where $\mathbf{1}[\cdot]$ is the indicator function.

Intuitively, this feature captures whether the source claim directly regards the target claim as one of its most similar semantic anchors. This is stronger than raw cosine similarity because it introduces a thresholded, rank-aware notion of neighborhood membership. A candidate edge whose target is explicitly retained in the source neighborhood is more likely to correspond to a meaningful discourse relation than a pair that merely has a moderate similarity score.

3.7.9 Feature 5: Source-in-Target Neighbors

The reciprocal membership feature is:

$$f_5(u, v) = \mathbf{1}[u \in N_k(v)] \tag{15}$$

This is not redundant with f_4 because the retained similarity graph is directed: v can be among the top-k neighbors of u without the reverse being true. This asymmetry matters because one claim can be semantically specific relative to another. The model can therefore exploit whether the similarity relationship is one-sided or mutual.

3.7.10 Feature 6: Neighbor-Set Jaccard Overlap

The overlap between the similarity neighborhoods of the source and target is measured with Jaccard similarity:

$$f_6(u, v) = \text{Jaccard}(N_k(u), N_k(v)) = \frac{|N_k(u) \cap N_k(v)|}{|N_k(u) \cup N_k(v)|} \tag{16}$$

with the convention that the value is 0 when the union is empty.

This feature measures whether the two claims live in the same local semantic region of the debate. Even if u and v are not direct nearest neighbors of one another, they may still share many neighbors. Such shared-neighborhood evidence can indicate common topical framing, parallel subarguments, or participation in the same semantic cluster. This is particularly valuable for discourse graphs where indirect neighborhood structure often matters as much as direct pair similarity.

3.7.11 Feature 7: Token-Level Jaccard Overlap

The final graph feature is a token-level Jaccard overlap. Let $T(u)$ and $T(v)$ denote the lowercase whitespace-token sets of the source and target claims. Then:

$$f_7(u, v) = \text{Jaccard}(T(u), T(v)) = \frac{|T(u) \cap T(v)|}{|T(u) \cup T(v)|} \tag{17}$$

This feature provides a simpler lexical overlap signal than full TF–IDF cosine similarity. It is useful because some relation cues come from literal lexical reuse, especially when one claim explicitly echoes or reframes another. It also offers robustness when TF–IDF weighting downweights common but discourse-relevant terms.

3.7.12 Relationship Between Jaccard Similarity and Jaccard Overlap

In this report, the terms *Jaccard similarity* and *Jaccard overlap* refer to the same set-based quantity:

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{18}$$

The distinction is interpretive rather than mathematical:

- for $N_k(u)$ and $N_k(v)$, it is a *neighbor-set overlap*,
- for $T(u)$ and $T(v)$, it is a *token-set overlap*.

3.7.13 Final Input Representation

The graph-aware classifier uses:

$$\mathbf{x}_{\text{full}} = [\mathbf{x}_{\text{NLI}} \; ; \; \mathbf{x}_{\text{stance}} \; ; \; \mathbf{x}_{\text{graph}}] \in \mathbb{R}^{13} \tag{19}$$

where:

$$\mathbf{x}_{\text{NLI}} \in \mathbb{R}^3, \qquad \mathbf{x}_{\text{stance}} \in \mathbb{R}^3, \qquad \mathbf{x}_{\text{graph}} \in \mathbb{R}^7$$

4 Experimental Setup

The quantitative results reported in this document are based on a saved benchmark run with the following configuration.

4.1 Configuration

Setting	Value
Corpus	`synthetic_corpus-001`
Split	`eval`
Max debates	8
Small mode	`true`
Test size	0.25
Seed	0
NLI model	`MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli`
Stance model	`krishnagarg09/stance-detection-semeval2016`
Similarity top-k	8
Similarity threshold	0.15

4.2 Train/Test Debate Split

Train debates:

- `node_link_data-debate-eval-0013`
- `node_link_data-debate-eval-0029`
- `node_link_data-debate-eval-0044`
- `node_link_data-debate-eval-0015`
- `node_link_data-debate-eval-0028`
- `node_link_data-debate-eval-0001`

Test debates:

- `node_link_data-debate-eval-0037`
- `node_link_data-debate-eval-0002`

4.3 Example Counts

Set	Example count
Training examples	7224
Test examples	2664

4.4 Class Distribution

Split	Support	Attack	Neutral	Total
Train	1806	1806	3612	7224
Test	666	666	1332	2664
Overall	2472	2472	4944	9888

4.5 Training Protocol

The benchmark uses the following training procedure:

1. load debates and construct candidate support, attack, and neutral examples,
2. split debates at the debate level rather than the row level,
3. compute base transformer features for every example,
4. compute graph features independently inside each debate,
5. fit the final classifier on the concatenated training representation,
6. evaluate on held-out debates only.

5 Architecture Diagram

5.1 Classifier Choice

The graph-aware model uses a Random Forest classifier:

- `n_estimators = 400`
- `random_state = 0`
- `n_jobs = -1`
- `class_weight = "balanced_subsample"`
- `min_samples_leaf = 2`

This is a deliberate contrast to the meta baseline, which uses linear multinomial logistic regression. The rationale is straightforward:

- the six transformer logits are approximately linearly useful,
- the graph features introduce threshold-like and interaction-heavy signals,
- Random Forests can model nonlinear interactions between lexical similarity, local degree patterns, and transformer confidence without requiring explicit manual feature crosses.

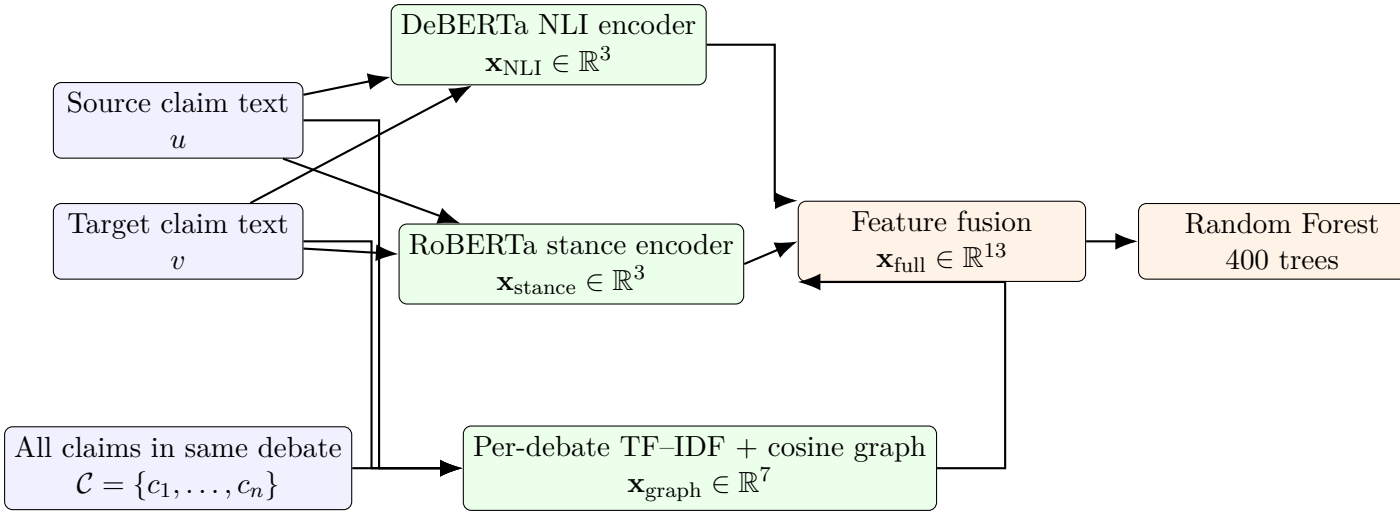


Figure 1: Revised architecture of the graph-aware stance model. The model fuses pairwise transformer evidence with debate-local graph-context features before nonlinear classification.

5.1.1 Decision Function of the Random Forest

Let the forest contain M decision trees, with $M = 400$ in the current implementation. For an input feature vector $\mathbf{x}_{\text{full}}$, each tree T_m outputs a class probability distribution over the three labels:

$$T_m(\mathbf{x}_{\text{full}}) \in [0, 1]^3$$

The forest prediction is the average probability vector:

$$\hat{\mathbf{p}}(\mathbf{x}_{\text{full}}) = \frac{1}{M} \sum_{m=1}^{M} T_m(\mathbf{x}_{\text{full}})$$

The final class prediction is:

$$\hat{y} = \arg \max_{c \in \{\texttt{support}, \texttt{attack}, \texttt{neutral}\}} \hat{p}_c(\mathbf{x}_{\text{full}})$$

This form is especially appropriate for the current feature space because it allows thresholding and feature interactions to emerge from the tree splits. For example, the model can learn that a moderate entailment logit is useful only when the target is also among the source's semantic neighbors, or that a high token overlap should be interpreted differently depending on neighborhood asymmetry.

5.2 Detailed Training Procedure

5.2.1 Step 1: Debate Sampling

The benchmark first loads debates from Syncialo and shuffles them with a fixed seed. In the saved experiment, the first eight debates from the shuffled order were used because small mode was active.

5.2.2 Step 2: Debate-Level Split

The system then splits at the debate level, not the row level. This is important because it prevents leakage from one debate into both train and test.

5.2.3 Step 3: Pair and Neutral Construction

For each debate:

1. include all gold support/attack edges,
2. sample non-edge pairs from the same debate,
3. label sampled non-edges as neutral,
4. convert each pair into (premise, hypothesis, label).

5.2.4 Step 4: Base Transformer Featurization

For each example, the system computes:

$$\mathbf{x}_{\text{base}} = [\ell_{\text{contradiction}}, \ell_{\text{neutral}}, \ell_{\text{entailment}}, \ell_{\text{against}}, \ell_{\text{none}}, \ell_{\text{favor}}]$$

5.2.5 Step 5: Graph Feature Construction

Graph features are constructed separately for each debate using the TF–IDF similarity graph. The resulting arrays are aligned with the same training and test examples.

5.2.6 Step 6: Model Training

The graph-aware classifier is trained on:

$$\mathbf{X}_{\text{train}} = [\mathbf{X}_{\text{train,base}} \ ; \ \mathbf{X}_{\text{train,graph}}]$$

5.2.7 Step 7: Evaluation

At test time, the system concatenates the test base features and test graph features and predicts one of the three labels.

6 Baselines Used for Comparison

The benchmark compares four systems.

6.1 DeBERTa Rule/Decision Baseline

This baseline uses the NLI model outputs directly and then maps them into relation decisions using an older rule-based decision layer. The system therefore has a two-stage structure: a transformer encoder produces contradiction, neutral, and entailment evidence, and a hand-designed decision rule converts those outputs into one of the three discourse labels.

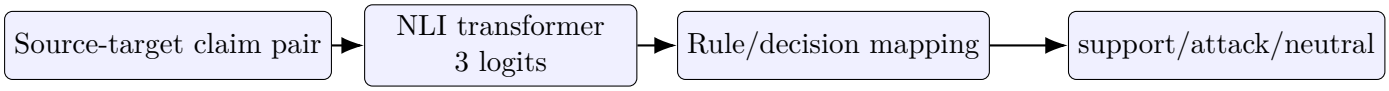


Figure 2: Architecture of the DeBERTa rule/decision baseline.

6.2 Stance Baseline

This baseline predicts the discourse label from the stance model alone. The target-dependent stance classifier produces logits corresponding to against, none, and favor, and these outputs are directly converted into the final stance-relation decision without any graph features or NLI evidence.

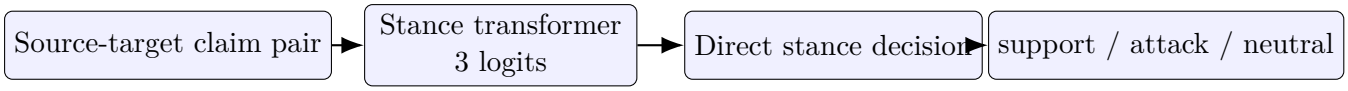


Figure 3: Architecture of the stance-only baseline.

6.3 DeBERTa + stance logistic regression Regression Baseline

This is the stronger non-graph baseline. It concatenates the three NLI logits and the three stance logits into a six-dimensional vector and then fits a multinomial logistic regression classifier over that fused representation.

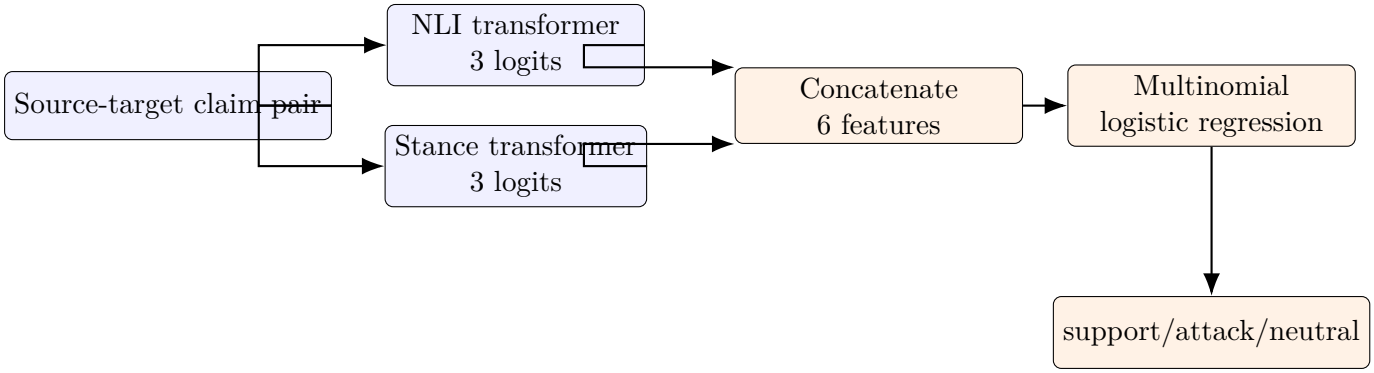


Figure 4: Architecture of the DeBERTa + stance logistic regression-regression baseline.

6.4 Graph-Aware Model

This is the proposed model. It uses the same six transformer features as the meta baseline, but augments them with seven debate-local graph features and swaps the linear meta-classifier for a Random Forest.

7 Results

7.1 Overall Metrics

Model	Accuracy	Macro-F1	Weighted-F1
DeBERTa baseline	0.5601	0.3907	0.4658
stance model(semeval2017) baseline	0.3517	0.3104	0.3349
DeBERTa + stance logistic regression regression	0.6513	0.6557	0.6477
Graph-aware model	**0.8818**	**0.8665**	**0.8811**

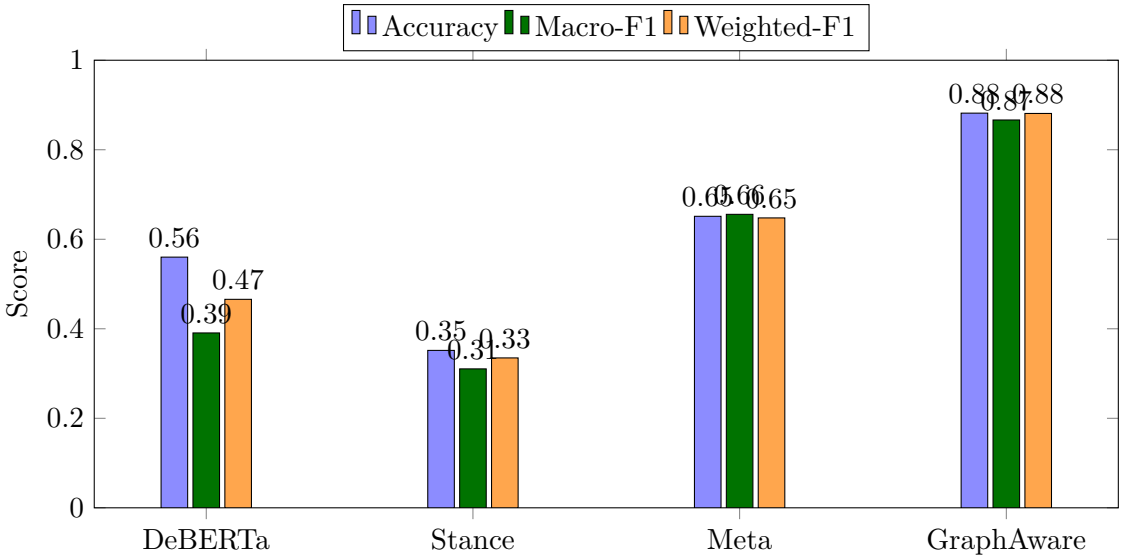


Figure 5: Comparison of the four evaluated models across the three primary aggregate metrics.

7.2 Metric Comparison Bar Charts

7.3 Per-Class Scores

Model	Class	Precision	Recall	F1
DeBERTa	Support	0.8182	0.1081	0.1910
	Attack	**0.8722**	0.1742	0.2904
	Neutral	0.5338	**0.9790**	0.6909
stance	Support	0.2607	0.7162	0.3822
	Attack	0.2844	0.0931	0.1403
	Neutral	0.6461	0.2988	0.4086
DeBERTa + stance logistic regression	Support	0.5726	0.7102	0.6340
	Attack	0.6257	0.8183	0.7092
	Neutral	0.7415	0.5383	0.6237
Graph-aware	Support	**0.8485**	**0.8243**	**0.8363**
	Attack	0.8525	**0.8243**	**0.8382**
	Neutral	**0.9111**	0.9392	**0.9250**

7.4 Confusion Matrices

Using label order [`support`, `attack`, `neutral`]:

DeBERTa baseline

$$\begin{bmatrix} 72 & 2 & 592 \\ 3 & 116 & 547 \\ 13 & 15 & 1304 \end{bmatrix}$$

stance baseline

$$\begin{bmatrix} 477 & 74 & 115 \\ 501 & 62 & 103 \\ 852 & 82 & 398 \end{bmatrix}$$

DeBERTa + stance logistic regression baseline

$$\begin{bmatrix} 473 & 39 & 154 \\ 25 & 545 & 96 \\ 328 & 287 & 717 \end{bmatrix}$$

Graph-aware model

$$\begin{bmatrix} 549 & 56 & 61 \\ 56 & 549 & 61 \\ 42 & 39 & 1251 \end{bmatrix}$$

7.5 Confusion Matrix Figures

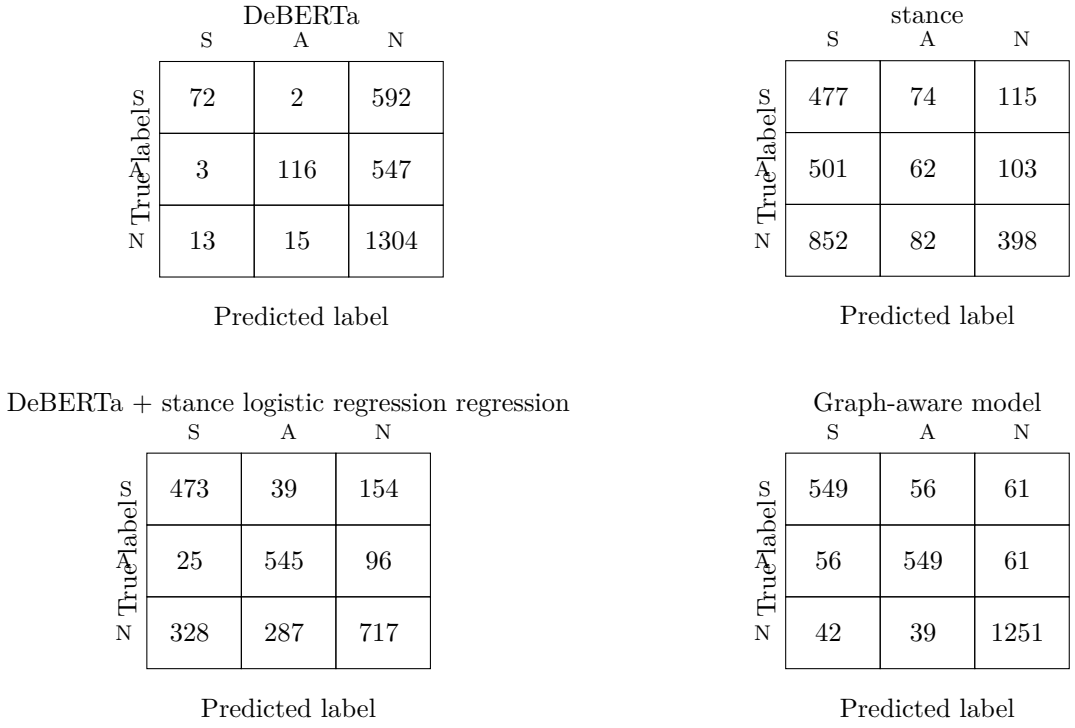


Figure 6: Confusion matrices for all four models. Labels use the order S = support, A = attack, N = neutral.

8 Interpretation of Results

8.1 Why the Graph-Aware Model Helps

The results suggest that the graph-aware model does not merely add noise or redundant features. Instead, it substantially improves both balanced performance and absolute accuracy. There are several likely reasons.

1. **Debate-local context reduces ambiguity.** Two claims with uncertain pairwise transformer evidence may still be easy to classify if they are very close in the local semantic neighborhood structure.

2. **The graph features help separate neutral edges.** The graph-aware model shows a large gain on the neutral class, which is often where pure pairwise systems are unstable.

3. **Nonlinear interactions matter.** The Random Forest can learn interactions such as:
   - high semantic similarity plus strong entailment-like NLI signal,
   - asymmetric neighborhood membership plus attack-biased stance logits,
   - strong local overlap but weak direct textual polarity.
4. **The improvements are broad, not class-specific.** The graph-aware model improves support, attack, and neutral simultaneously.

8.2 Comparison to the Models

The older direct baselines show a typical pattern:

- the NLI-only path overpredicts neutral,
- the stance-only path is unstable across classes,
- the DeBERTa + stance logistic regression model already gives a large boost by combining the two transformer sources,
- the graph-aware model adds another large jump by injecting debate-local structure.

8.3 Comparison to the DeBERTa + stance logistic regression Baseline

The graph-aware model improves over the meta baseline by:

$$\Delta\text{Accuracy} = 0.8818 - 0.6513 \approx 0.2305$$
$$\Delta\text{Macro-F1} = 0.8665 - 0.6557 \approx 0.2108$$

This is a very large improvement. It strongly indicates that, at least on the saved benchmark split, graph-aware local context is highly informative.

9 Limitations and Caveats

9.1 The Current Benchmark Is Small

The saved quantitative results come from the small benchmark configuration:

- only eight debates,
- six train debates,
- two test debates.

These results are promising, but they should not be treated as the final word on generalization. A larger benchmark should be run before making strong performance claims.

9.2 The Similarity Graph Is Lexical, Not Neural

The graph-aware features are built from TF–IDF similarity, not learned graph embeddings or Graph Neural Networks. This has two implications:

- it is simple, fast, and interpretable,
- it may miss deeper semantic relations that lexical overlap cannot capture.

9.3 The Model Is Graph-Aware, Not a Full GNN

Despite the name, the current model is not a message-passing neural graph model. It is better described as:

> a stacked transformer-plus-graph-feature classifier.

This is still valuable, but conceptually it is a feature-augmented tabular model rather than a deep graph network.

9.4 Neutral Sampling Matters

Neutral labels are generated by non-edge sampling. Performance on the neutral class is therefore influenced by the sampling strategy and may change if harder negatives are introduced.

9.5 Experimental Configuration Dependence

Different runs can use different subsets of debates, different sampling seeds, and different feature-cache states. Consequently, trained graph-aware models should be compared only when the underlying experimental configuration is controlled.

10 Next Steps

The current graph-aware model is a strong intermediate design. The next meaningful upgrades would be:

1. run the benchmark on many more debates and multiple seeds,
2. add hard-negative mining rather than purely random non-edge sampling,
3. compute feature importance for the Random Forest,
4. compare TF–IDF graph features with embedding-based nearest-neighbor graph features,
5. test gradient boosting or calibrated tree ensembles,
6. move toward iterative graph refinement or a true GNN if larger-scale training becomes feasible.

11 Conclusion

The graph-aware model studied here is a practical and effective extension of a transformer-based stance pipeline. It combines:

- transformer-based textual reasoning,
- debate-local graph context,
- a nonlinear final classifier.

On the current saved Syncialo benchmark split, it substantially outperforms:

- the older direct NLI baseline,
- the older direct stance baseline,
- the DeBERTa + stance logistic regression-regression baseline.

Its main contribution is not merely stronger classification accuracy, but a tighter alignment between stance prediction and discourse graph structure. That makes it especially suitable for a system whose downstream goals include graph export, visualization, clustering, centrality analysis, and broader discourse interpretation.